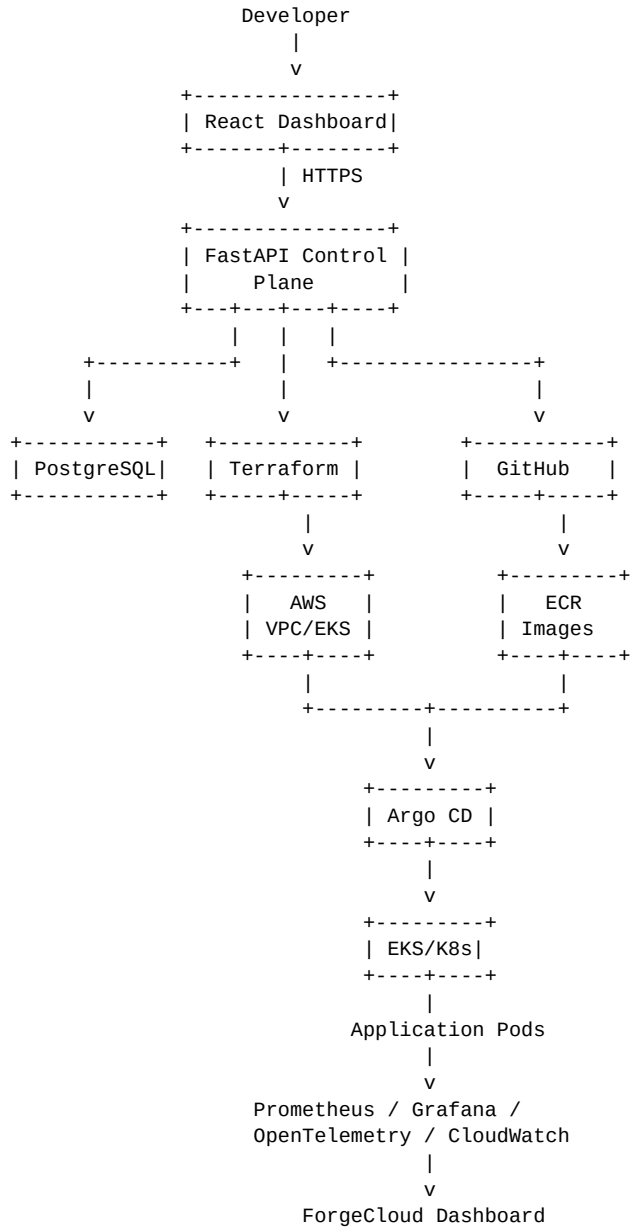


ForgeCloud Complete System Design

End-to-end architecture, requirements, layers and engineering principles

1. Complete System Architecture



2. Architectural Layers

Layer	Components
Presentation	React dashboard
Control plane	FastAPI services and REST APIs
Data	PostgreSQL
Automation	Terraform, GitHub Actions
Artifact	Docker, Amazon ECR

Layer	Components
Runtime	Kubernetes / Amazon EKS
GitOps	Argo CD
Observability	Prometheus, Grafana, OpenTelemetry, CloudWatch
Security	JWT, RBAC, IAM, Secrets, network controls

3. Functional Requirements

- Users can authenticate and receive role-appropriate access.
- Developers can create and manage application metadata.
- The platform can initiate deployment workflows.
- Applications can run as containers in Kubernetes/EKS.
- Deployment history and status are visible.
- Monitoring data can be viewed through the platform.
- Applications can scale based on resource utilization.
- Failed releases can be rolled back.
- Infrastructure is defined through Terraform.
- Operations are recorded for auditing.

4. Non-Functional Requirements

- Security: least privilege and protected credentials.
- Reliability: health checks, replacement and rollback.
- Observability: metrics, logs and traces.
- Maintainability: modular services and versioned infrastructure.
- Reproducibility: Terraform and declarative Kubernetes.
- Testability: automated unit/API/integration validation.
- Cost awareness: small practical AWS resources for a student project.

5. System Design Principle

Every technology must have a reason, every architectural decision should be explainable, and important claims should be supported by measurements from testing.